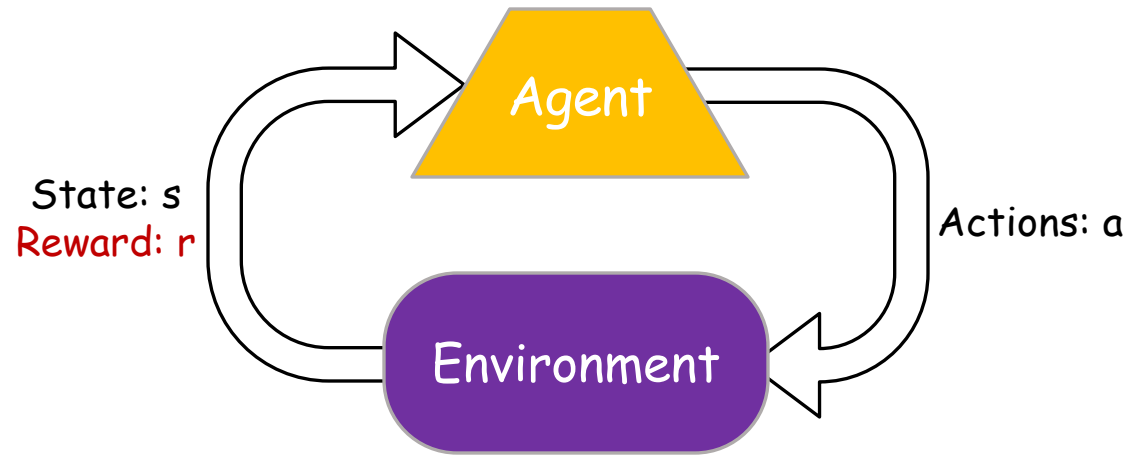


Reinforcement Learning

CHAPTER 21

Adapted from slides by Dan Klein, Pieter Abbeel, David Silver, and Raj Rao

Reinforcement Learning



□ Basic idea:

- Receive feedback in the form of **rewards**
- Agent's utility is defined by the reward function
- Must (learn to) act so as to **maximize expected rewards**
- All learning is based on observed samples of outcomes!

Examples

- ❑ <https://www.youtube.com/watch?v=N3fT7N2USmA>
- ❑ <https://www.youtube.com/watch?v=V1eYniJ0Rnk&index=31&list=P L5nBAYUyJTrM48dViibyi68urttMIUv7e>

Reinforcement Learning

❑ Still assume a Markov decision process (MDP):

- A set of states $s \in S$
- A set of actions (per state) A
- A model $T(s, a, s')$
- A reward function $R(s, a, s')$

❑ Still looking for a policy $\pi(s)$

❑ New twist: don't know T or R

- We don't know which states are good or what the actions do
- Must actually try actions and states out to learn

Passive Reinforcement Learning

❑ Simplified task: policy evaluation

- Input: a fixed policy $\pi(s)$
- You don't know the transitions $T(s, a, s')$
- You don't know the rewards $R(s, a, s')$
- Goal: learn the state values - V^π

❑ In this case:

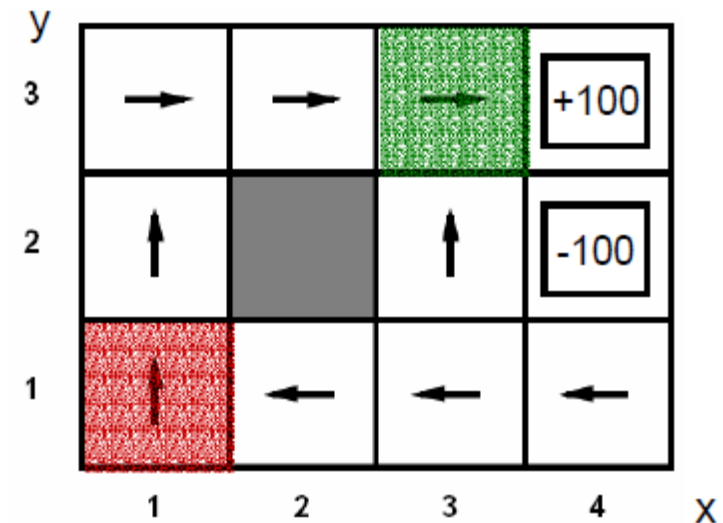
- No choice about what actions to take
- Just execute the policy and learn from experience
- **This is NOT offline planning! You actually take actions in the world.**

Episodes/Trials

Episodes/trials

(1,1) up -1	(1,1) up -1
(1,2) up -1	(1,2) up -1
(1,2) up -1	(1,3) right -1
(1,3) right -1	(2,3) right -1
(2,3) right -1	(3,3) right -1
(3,3) right -1	(3,2) up -1
(3,2) up -1	(4,2) exit -100
(3,3) right -1	(done)
(4,3) exit +100	
(done)	

Empirical Policy Evaluation



$$\gamma = 1, R = -1$$

Method 1 - Adaptive Dynamic Programming

❑ Idea

- Learn the model empirically
- Solve the MDP as if the learned model were correct
- Model based learning

❑ Empirical model learning

- Simplest case
 - Count the outcomes for each s, a
 - Normalize to give estimate of $T(s, a, s')$
 - Discover $R(s, a, s')$ the first time we experience (s, a, s')
- More complex learners are possible, if more domain knowledge is available

Method 1 - Adaptive Dynamic Programming (2)

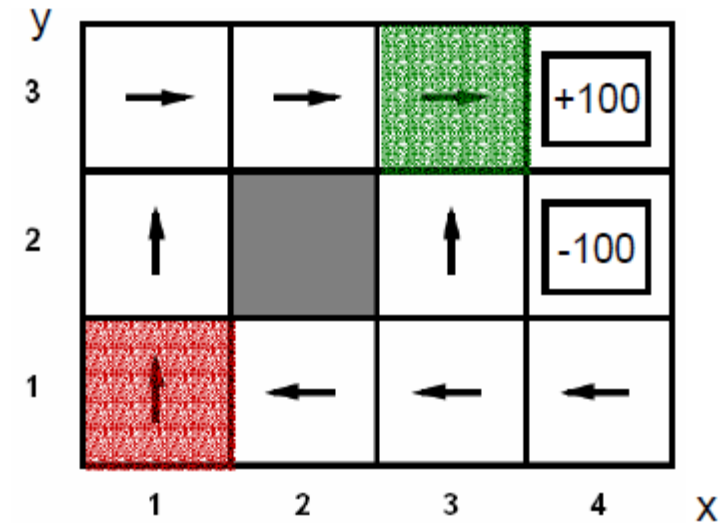
- ❑ Estimate the transitions $T(s, \pi(s), s')$ and rewards $R(s, \pi(s), s')$ from the episodes
- ❑ Provides a model for MDP
- ❑ Apply Bellman equations to evaluate the policy

$$V_0^\pi(s) = 0$$
$$V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_k^\pi(s')]$$

Method 1 - Adaptive Dynamic Programming (3)

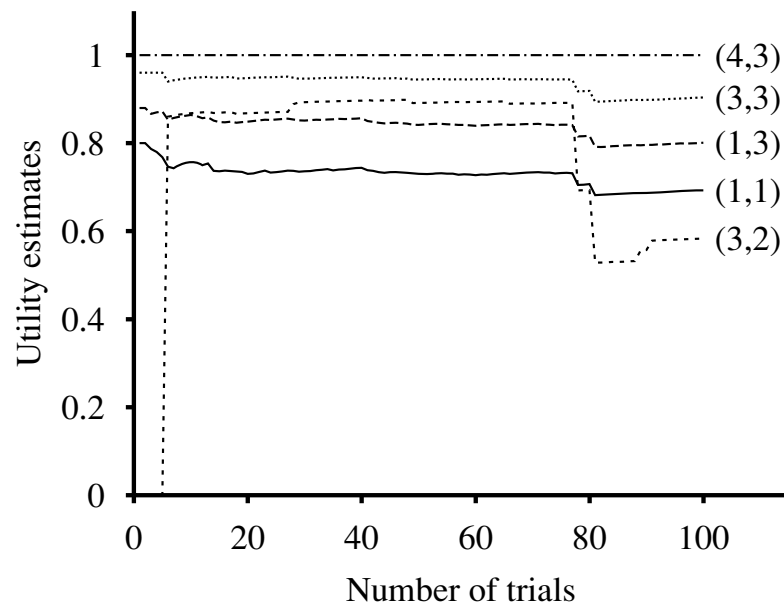
❑ Episodes

(1,1) up -1	(1,1) up -1
(1,2) up -1	(1,2) up -1
(1,2) up -1	(1,3) right -1
(1,3) right -1	(2,3) right -1
(2,3) right -1	(3,3) right -1
(3,3) right -1	(3,2) up -1
(3,2) up -1	(4,2) exit -100
(3,3) right -1	(done)
(4,3) exit +100	
(done)	

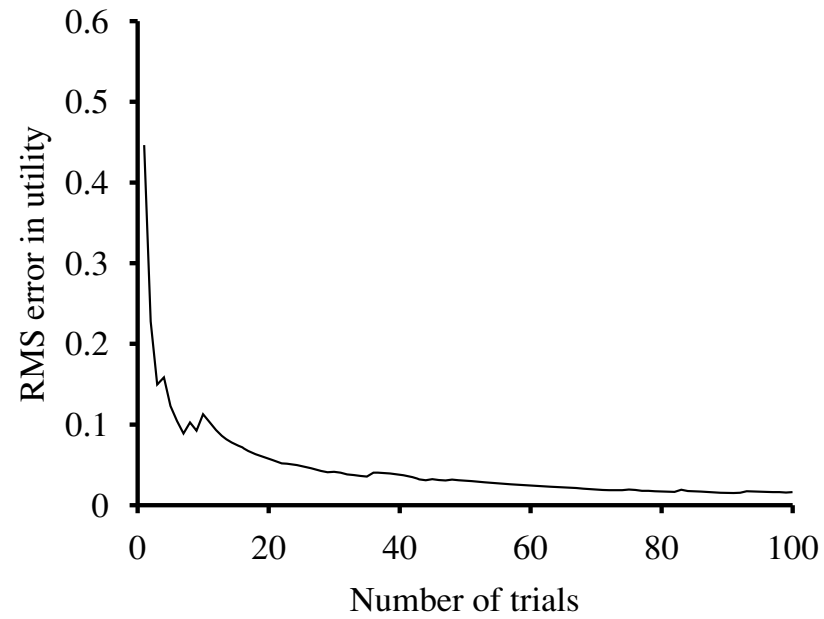


$\gamma = 1, R = -1$

Method 1 - Adaptive Dynamic Programming (4)



(a)



(b)

Method 1 - Adaptive Dynamic Programming (5)

- ❑ Intractable for large state spaces
- ❑ The ADP agent is limited only by its ability to learn the transition model.
 - ADP agent acts as if the learned model is correct – need not always be true.
- ❑ Can we turn it into a model free approach?

Method 2 - Direct Utility Estimation

- ❑ Goal: Compute values for each state under π
- ❑ Idea: Average together observed sample values
 - Act according to π
 - Every time you visit a state, write down what the sum of discounted rewards turned out to be
 - Average those samples
- ❑ This is called direct utility estimation/evaluation
 - Model free learning

Method 2 – Direct Utility Estimation

❑ Monte-Carlo Policy Evaluation

❑ To estimate $V^{\pi}(s)$

❑ Loop over episodes

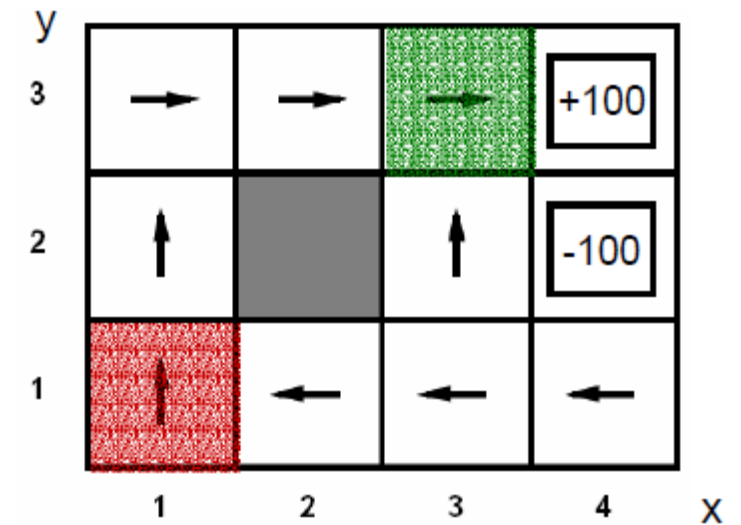
- The first time-step t that state s is visited in an episode
- Increase the counter $N(s)$ - number of times s is visited in the episodes (can be multiple times in an episode)
- Total return $T(s)$ – sum of the returns from each visit
- Estimated return $\hat{V}^{\pi}(s) = T(s)/N(s)$
 - As $N(s) \rightarrow \infty$, $\hat{V}^{\pi}(s) \rightarrow V^{\pi}(s)$

Method 2- Direct Utility Estimation

Episodes/trials

(1,1) up -1	(1,1) up -1
(1,2) up -1	(1,2) up -1
(1,2) up -1	(1,3) right -1
(1,3) right -1	(2,3) right -1
(2,3) right -1	(3,3) right -1
(3,3) right -1	(3,2) up -1
(3,2) up -1	(4,2) exit -100
(3,3) right -1	(done)
(4,3) exit +100	
(done)	

Empirical Policy Evaluation



$$\gamma = 1, R = -1$$

Incremental Mean

□ The mean $\mu_1, \mu_2, \dots$ of a sequence $x_1, x_2, \dots$ can be computed incrementally

$$\begin{aligned}\mu_k &= \frac{1}{k} \sum_{i=1}^k x_i \\ &= \frac{1}{k} (x_k + \sum_{i=1}^{k-1} x_i) \\ &= \frac{1}{k} (x_k + (k-1)\mu_{k-1}) \\ &= \mu_{k-1} + \frac{1}{k} (x_k - \mu_{k-1})\end{aligned}$$

Method 3 – Temporal Difference Learning

❑ Idea – Learn from every experience

- Update $V(s)$ each time we experience a transition (s, a, s', r)
- Likely outcomes s' will contribute to updates more often

❑ Temporal difference learning of values

- Policy still fixed, still doing evaluation
- Model-free – no knowledge of the MDP
- Learns from incomplete episodes, bootstrapping
 - Update a guess towards a guess

Method 3 – Temporal Difference Learning

❑ Idea – Learn from every experience

- Update $V(s)$ each time we experience a transition (s, a, s', r)
- Likely outcomes s' will contribute to updates more often

❑ Temporal difference learning of values

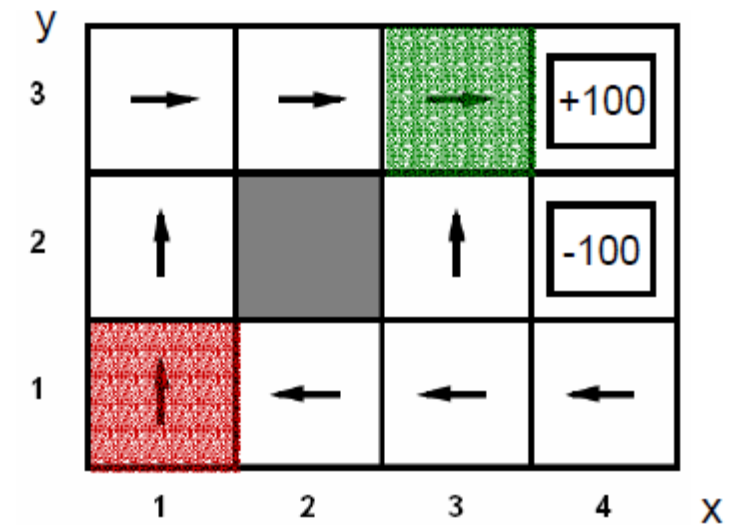
- Policy still fixed, still doing evaluation
- $sample = R(s, \pi(s), s') + \gamma \hat{V}^\pi(s')$ - TD target
- $\hat{V}^\pi(s) \leftarrow \hat{V}^\pi(s) + \alpha (sample - \hat{V}^\pi(s))$
 - $sample - \hat{V}^\pi(s)$ - TD error

Method 3 - Temporal Difference Learning

❑ Episodes/trials

(1,1) up -1	(1,1) up -1
(1,2) up -1	(1,2) up -1
(1,2) up -1	(1,3) right -1
(1,3) right -1	(2,3) right -1
(2,3) right -1	(3,3) right -1
(3,3) right -1	(3,2) up -1
(3,2) up -1	(4,2) exit -100
(3,3) right -1	(done)
(4,3) exit +100	
(done)	

Empirical Policy Evaluation



$$\gamma = 1, R = -1$$

TD and ADP

- ❑ Both try to make local adjustments to utility estimates
- ❑ TD adjusts a state to agree to its observed successor
- ❑ ADP adjusts the state to agree to with all the successors that might occur, weighted by their probabilities
 - TD adjustments averaged over a large number of transitions will result in the same outcome

Active Reinforcement Learning

❑ Passive learning agent

- Simplified task: policy evaluation
 - Input: a fixed policy $\pi(s)$
 - You don't know the transitions $T(s, a, s')$
 - You don't know the rewards $R(s, a, s')$
 - Goal: learn the state values

❑ Active learning agent

- Full reinforcement learning
 - You don't know the transitions $T(s, a, s')$
 - You don't know the rewards $R(s, a, s')$
 - You choose the actions now
 - Goal: learn the optimal policy/values

❑ The Learning agent makes choices.

Modified ADP Agent (1)

❑ Model based learning

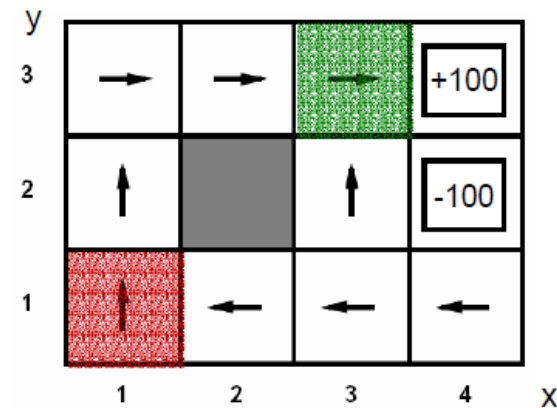
- Learn the model empirically
- Solve the MDP as if the learned model were correct

❑ Rather than learning the model for a fixed policy - Learn the model for all actions

❑ Value or Policy iteration for a given model

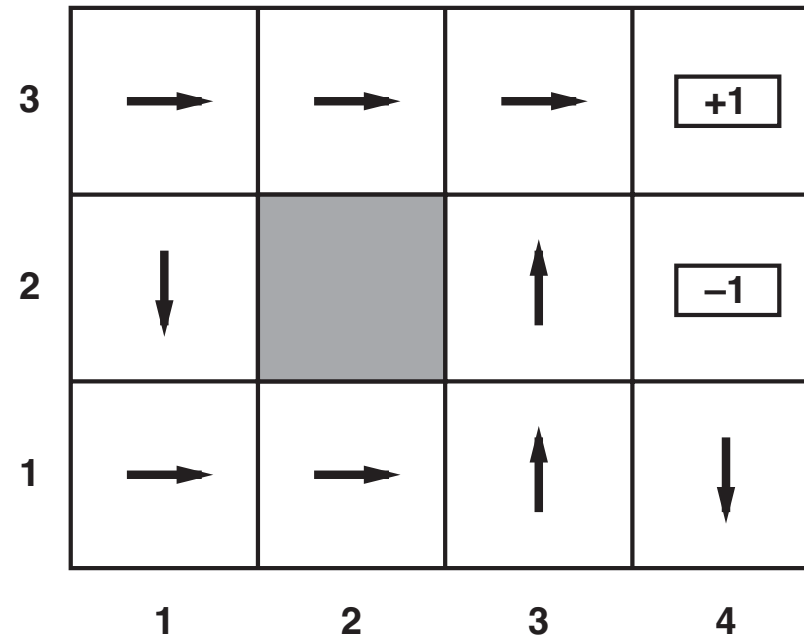
- Optimal values and policies

❑ Is this a good option?



Modified ADP Agent (2)

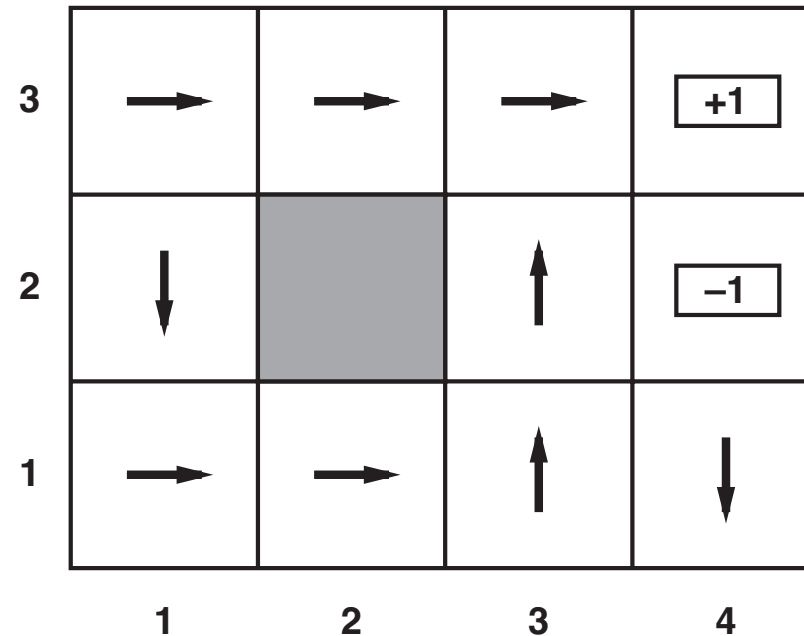
- ❑ Imagine the agent found the lower path to the good exit first
- ❑ Some states will never be visited following this policy from (1,1)
- ❑ The agent will keep reusing this policy
 - Never learns about better regions of the space if current policy neglects them - Exploitation



Modified ADP Agent (3)

❑ Fundamental Tradeoff between exploitation and exploration

- Exploration – must take actions with suboptimal estimates to discover new rewards and increase eventual utility
- Exploitation – once the true optimal policy is learned, exploration reduces utility
- Agents explore in the beginning and exploit in the limit



Modified ADP Agent (4)

❑ Is there an Optimal exploration policy?

- Field of statistical decision theory – Bandit problems

❑ GLIE Scheme – Greedy in the limit of infinite exploration

❑ Examples

- Choose a random action a fraction $1/t$ of the time and to follow greedy policy otherwise
- Give weights to actions that the agent has not tried very often, while tending to avoid actions that are believed to be of low utility.

Exploration function

□ Modify the Bellman equation

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

- To assign higher utility to unexplored state-action pairs.

- Acts like an optimistic prior of the utility

- V^+ - optimistic estimate of the utility

- $N(s, a)$ - number of times action a has been tried in state s

$$V^+(s) \leftarrow \max_a f \left(\sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^+(s')], N(s, a) \right)$$

- $f(v, n)$ - exploration function

- Tradeoff between greed and curiosity

Exploration/Exploitation

□ How to force exploration?

○ Simplest – random actions (greedy)

- Every time flip a coin
- With probability p , act randomly
- With probability $1-p$, act according to the current policy

○ Problem with random actions

- Lot of unnecessary random actions once learning is complete
- Lower probability over time

Exploration function

$$f\left(\sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^+(s')], N(s, a)\right)$$

❑ f should be increasing in v

- Preference for high values of v (greed)

❑ f should be decreasing in n

- Preference for actions that have not been tried often (curiosity)

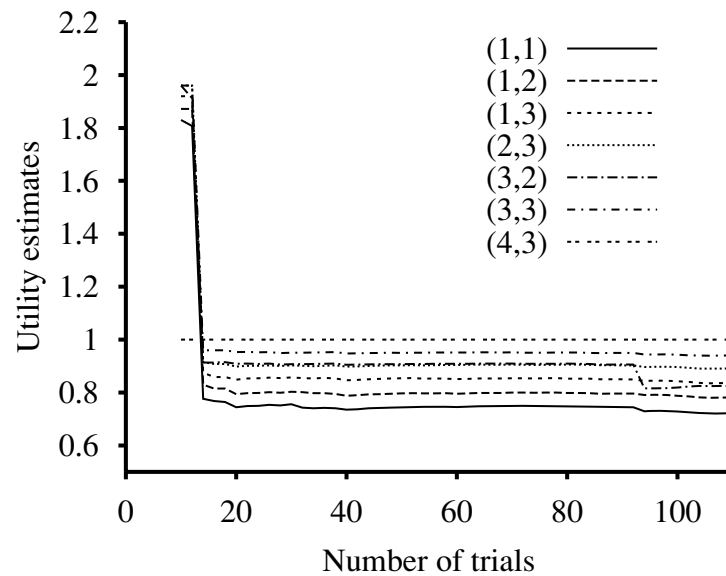
❑ Example

$$f(v, n) = \begin{cases} R^+, & \text{if } n < N_e \\ v, & \text{otherwise} \end{cases}$$

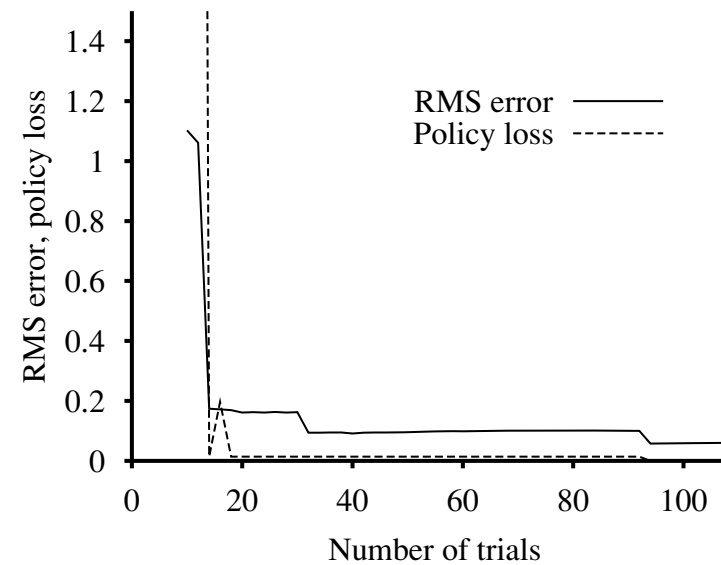
- Where R^+ is an optimistic estimate of the best possible reward obtainable in any state and N_e is a fixed parameter

Exploration function

□ $R^+ = 2, N_e = 5$



(a)



(b)

Temporal Difference Learning

❑ Model free way to do policy evaluation

- $sample = R(s, \pi(s), s') + \gamma \hat{V}^\pi(s')$
- $\hat{V}^\pi(s) \leftarrow \hat{V}^\pi(s) + \alpha (sample - \hat{V}^\pi(s))$

❑ For obtaining a new policy

$$\pi(s) = \arg \max_a Q(s, a)$$

$$Q(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]$$

❑ So Q Learning

Q-value Iteration

❑ Value iteration: find successive (depth-limited) values

- Start with $V_0(s) = 0$,
- Given V_k calculate the depth $k+1$ values for all states

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

❑ But Q-values are more useful, so compute them instead

- Start with $Q_0(s, a) = 0$, which we know is right
- Given Q_k , calculate the depth $k+1$ q-values for all q-states:

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$
$$V_k(s') = \max_{a'} Q(s', a')$$

Q-Learning

❑ Q-Learning: sample-based Q-value iteration

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma \max_{a'} Q(s', a') \right]$$

❑ Learn $Q(s, a)$ values as you go

- Receive a sample (s, a, s', r)
- Consider your old estimate: $Q(s, a)$
- Consider your new sample estimate:
- $sample = R(s, a, s') + \gamma \max_{a'} Q(s', a')$



❑ Incorporate the new estimate into a running average:

- $Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha sample$

Q-Learning Properties

- ❑ Will converge to optimal policy even if the agent is acting sub optimally
 - If explored enough
 - If you make the learning rate small enough
- ❑ Under certain conditions
 - The environment model does not change
 - States and actions are finite
 - Rewards are bounded
- ❑ Also called as off-policy learning

SARSA

- ❑ The update rule is applied at the end of each (s, a, r, s', a') quintuplet.

$$Q(s, a) \leftarrow Q(s, a) + \alpha(R(s) + \gamma Q(s', a') - Q(s, a))$$

- ❑ For a greedy agent (always picks the best action), both Q-learning and SARSA are identical
- ❑ Q-learning pays no attention to the actual policy being followed
 - Always picks the best Q-value
- ❑ SARSA- on-policy algorithm

Generalization in Reinforcement Learning

❑ Q-learning keeps a table of all q-values

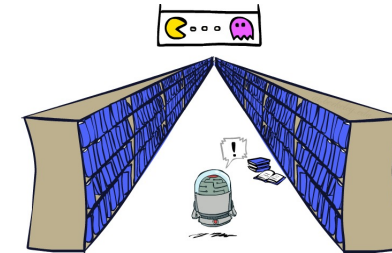
- Works well for small state spaces

❑ Real-world problems contain very large number of states - 10^{20} or 10^{40}

- Does not make sense to suppose that the agent must visit all the states to learn the optimal policy
- Too many states to hold Q-tables in memory

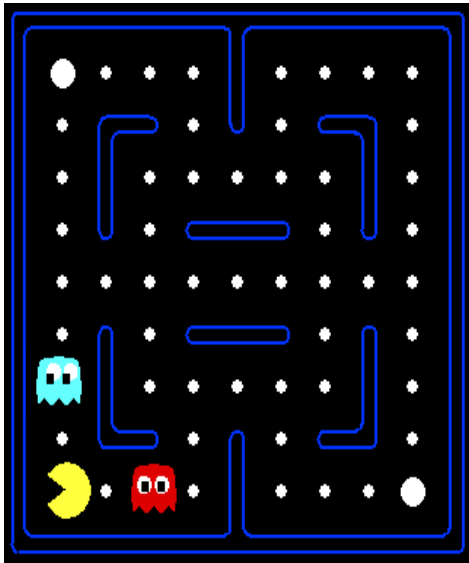
❑ Instead, we want to generalize

- Learn about a small number of training states from experience
- Generalize that experience to new, similar situations
- Which is?

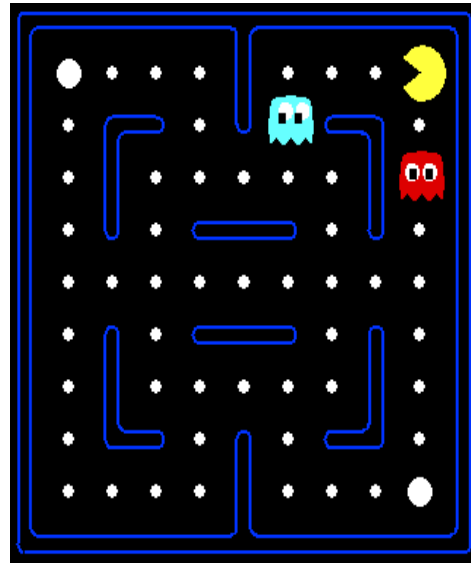


Example

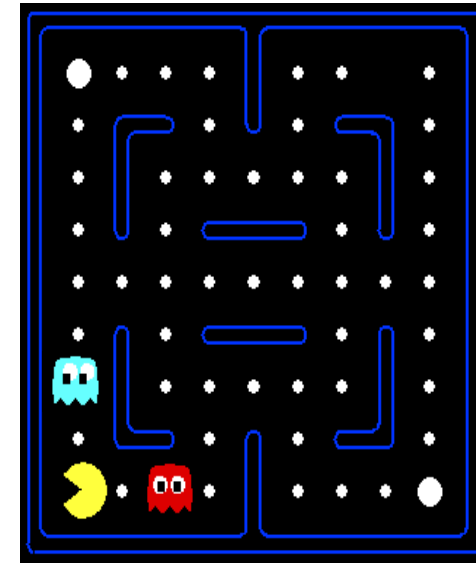
Let's say we discover through experience that this state is bad:



In naïve q-learning, we know nothing about this state:



Or even this one!



Feature-Based Representations

❑ Function Approximation

- Use some sort of representation for the Q-function other than a lookup table.

❑ Describe a state using a vector of features (properties)

- Features are functions from states to real numbers (often 0/1) that capture important properties of the state
- Example
 - Distance to the closest ghost
 - Distance to the closest dot
 - Number of ghost
 - Is pacman in a tunnel?
- We can also describe the Q-state function (or the value function) in terms of these features.

Function Approximations

- ❑ Using feature representation, we can write the q function (or the value function) for a state using a few parameters

$$V(s) = \theta_1 f_1(s) + \dots + \theta_N f_N(s)$$
$$Q(s, a) = \theta_1 f_1(s, a) + \dots + \theta_N f_N(s, a)$$

- ❑ Here we have selected linear functions
- ❑ Advantage is that the experience is summed up with a few parameters
- ❑ Disadvantage is that states may share the similar features, but have different values

Direct Utility Estimation Through Function Approximation

- ❑ Suppose that the utilities for the grid world can be defined as

$$\hat{V}_{\theta}(x, y) = \theta_0 + \theta_1 x + \theta_2 y$$

- ❑ Given a collection of episodes, we obtain the sample values

$$\hat{V}_{\theta}(x, y)$$

- And so the objective is to find parameters $(\theta_1, \theta_2, \theta_3)$ that best fits the observations aka linear regression!

- ❑ In the RL setting, the learning can be online as and when an episode is generated - online least squares regression

Online least squares

- ❑ If $(\theta_1, \theta_2, \theta_3) = (0.5, 0.2, 0.1)$, then $\hat{V}_\theta(1,1) = 0.8$
- ❑ Suppose the total reward obtained from the current trial is $v(1,1) = 0.4$
- ❑ Adjust the parameters such that the error reduces, where the error function is
 - $E(s) = \frac{1}{2} \left(\hat{V}_\theta(x, y) - v(x, y) \right)^2$
- ❑ And so the parameter update will be

Online least squares

- ❑ If $(\theta_1, \theta_2, \theta_3) = (0.5, 0.2, 0.1)$, the $\hat{V}_\theta(1,1) = 0.8$
- ❑ Suppose the total reward obtained from the current trial is $v(1,1) = 0.4$
- ❑ Adjust the parameters such that the error reduces, where the error function is
 - $E(s) = \frac{1}{2} \left(\hat{V}_\theta(x, y) - v(x, y) \right)^2$
- ❑ And so the parameter update will be
 - $\theta_i \leftarrow \theta_i - \alpha \frac{\partial E}{\partial \theta_i} = \theta_i + \alpha \left(v(x, y) - \hat{V}_\theta(x, y) \right) \frac{\partial \hat{V}_\theta(x, y)}{\partial \theta_i}$
 - $\theta_i \leftarrow \theta_i + \alpha \left(v(x, y) - \hat{V}_\theta(x, y) \right) f_i(x, y)$
 - Also referred to as Widrow-Hoff or Delta rule for online least squares

Direct Utility Estimation Through Function Approximation

- ❑ Suppose that the utilities for the grid world can be defined as

$$\hat{V}_{\theta}(x, y) = \theta_0 + \theta_1 x + \theta_2 y$$

- ❑ Given a collection of episodes, we obtain the sample values

$$\hat{V}_{\theta}(x, y)$$

- And so the objective is to find parameters $(\theta_1, \theta_2, \theta_3)$ that best fits the observations aka linear regression!

- ❑ In the RL setting, the learning can be online as and when an episode is generated - online least squares regression

- ❑ Changing the parameters θ in response to a single observation changes the values of the utility approximation function for every state!

- In this sense we achieve generalization.

Function Approximation to TD Learning!

- ❑ Adjust the parameters to reduce the temporal difference between successive states
- ❑ $\theta_i \leftarrow \theta_i + \alpha [R(s, \pi(s), s') + \gamma \hat{v}^\pi(s') - \hat{V}_\theta(s)] \frac{\partial \hat{V}_\theta(s)}{\partial \theta_i}$
- ❑ And thus, can also be extended to Q-Learning

Q – Learning through Function Approximations

$$Q(s, a) = \theta_1 f_1(s, a) + \dots + \theta_N f_N(s, a)$$

- ❑ Give the current estimates of q-values for all state, action pairs, and a new transition – (s, a, r, s')
- ❑ Difference = $R + \gamma \max_{a'} Q(s', a') - Q(s, a)$
- ❑ Traditional Q learning
 - $Q(s, a) \leftarrow Q(s, a) + \alpha[\text{difference}]$
- ❑ Under function approximation
 - $\theta_i \leftarrow \theta_i + \alpha[\text{difference}] f_i(s, a)$
- ❑ Intuitive Interpretation
 - Adjust the weights of active features

Deep Q-Learning

□ Q-function approximation extend to deep neural networks!

○ $Q(s, a; \theta)$ = deep neural network defined by θ

Remember: want to find a Q-function that satisfies the Bellman Equation:

$$Q^*(s, a) = \mathbb{E}_{s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q^*(s', a') | s, a \right]$$

Forward Pass

Loss function: $L_i(\theta_i) = \mathbb{E}_{s, a \sim \rho(\cdot)} [(y_i - Q(s, a; \theta_i))^2]$

where $y_i = \mathbb{E}_{s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) | s, a \right]$

Iteratively try to make the Q-value close to the target value (y_i) it should have, if Q-function corresponds to optimal Q^* (and optimal policy π^*)

Backward Pass

Gradient update (with respect to Q-function parameters θ):

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{s, a \sim \rho(\cdot); s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i) \right] \nabla_{\theta_i} Q(s, a; \theta_i)$$

Deep Q-Learning

[Mnih et al. NIPS Workshop 2013; Nature 2015]

Case Study: Playing Atari Games



Objective: Complete the game with the highest score

State: Raw pixel inputs of the game state

Action: Game controls e.g. Left, Right, Up, Down

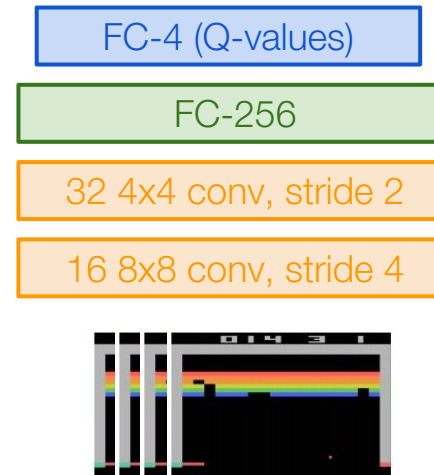
Reward: Score increase/decrease at each time step

Deep Q-Learning

[Mnih et al. NIPS Workshop 2013; Nature 2015]

Q-network Architecture

$Q(s, a; \theta)$:
neural network
with weights θ



Current state s_t : 84x84x4 stack of last 4 frames
(after RGB->grayscale conversion, downsampling, and cropping)

Policy Search

- ❑ Often feature-based policies that work well (win games, maximize utilities) aren't the ones that approximate V / Q best
 - Q-learning's priority: get Q-values close (modeling)
 - Action selection priority: get ordering of Q-values right (prediction)
- ❑ Why not directly search for a policy?
 - Learn policies that maximize rewards, not the values that predict them
- ❑ Parameterized representations of policy π
 - Similar to hill climbing

Summary

❑ Reinforcement Learning

❑ Passive learning

- ADP – Model based learning
- Monte-Carlo Approaches
 - Incremental MC learning
- TD learning

❑ Active Learning

- Modified ADP
- Modified TD aka Q learning

❑ Function Approximations

❑ Policy Search